API Gateway Pattern

Category: Edge / routing **One-liner:** A single entry point that sits in front of all microservices, routing, aggregating, securing, and shaping requests so clients never talk to services directly.

Lesson 1 — Introduction to the API Gateway Pattern

The fast-food counter

You order a burger at the counter: you order, pay and collect there, and never walk into the kitchen or to a separate billing desk. The counter is the gateway — **the single point at which the customer interacts with all the restaurant's services**.

What the pattern actually is

In a **microservice architecture** (many small services, each running separately and owning one business capability), the API Gateway is **a single entry point for external consumers** to reach all of them. Three independent benefits follow:

1. **Easier to manage** — one place where request-handling policy lives.
2. **Secures your services** — services need not be individually exposed.
3. **A unified interface to the outside world** — clients see one coherent API, not the seams between your teams.

Why the pattern exists

The shift from **monoliths** (one big unit) to microservices brought scalability and flexibility, but its biggest new problem is communication: **how do clients and services communicate efficiently across many services, and how do you manage the growing complexity of their interactions?** The gateway fills that gap.

The Netflix example

Opening Netflix on a TV needs the movie list, your preferences, subtitles and more — each from a separate microservice.

- **Without a gateway:** the TV app calls each service itself, so it must know each one exists, where it lives, how to **authenticate** to it (prove identity), and its response shape.
- **With a gateway:** the app makes one call; the gateway talks to whichever services are needed.

Note *which* client benefits most: TV and mobile apps can't be force-updated, so any **service topology** (which services exist and where) baked into them is frozen in the field for years.

The master orchestrator

The gateway does three things in sequence:

1. Serves as the **single entry point** for all client requests
2. **Routes** them to the appropriate microservices
3. **Gathers the responses** and returns one **unified response**

Its value is **masking the underlying complexity**: the client doesn't know how many services were involved or how they talk.

The headline advantages

- **Simplified client** — one gateway instead of many services.
- **Security** — **authentication** (who is calling) and **authorization** (what they may do) at one point.
- **Performance** — **caching**, response compression, and **load balancing** across service copies (§ Caching, § Load balancers).
- **Cross-cutting concerns** (needs every service shares) — logging, analytics and similar, handled once.

Lesson 2 — Advantages: the fifteen problems it solves

Each is a distinct interview argument:

#	Problem without a gateway	Gateway's answer
1	**Complexity of interactions** — every service talking to every other; connections explode	Central hub: a mesh of connections collapses into one point
2	**Inconsistent service calls** — different interaction styles and **data formats** (JSON, XML, binary); clients implement several dialects	One consistent interface; **translates calls and formats** so the client sees uniformity the **backend** lacks
3	**Security and authentication** per service — duplicated, easily inconsistent	Authenticate and authorize **centrally**, before anything reaches a service
4	**Load balancing** — some services get far more traffic	Distributes traffic so **no single service is overwhelmed**
5	**Performance** — network calls between scattered services add delay	**Caches responses**, reducing repeated calls
6	**Cross-cutting concerns** (logging, monitoring, analytics) in every service	Handled once at the gateway
7	**Legacy systems** needing awkward access	Talks to legacy behind a **modern interface**; awkwardness stops at the boundary
8	**Version management** — clients need different service versions (acute for un-updatable mobile/TV)	Routes to the right version per client
9	**Aggregation** — data spread over several services means several calls, painful on mobile	Aggregates into a **single consolidated response**
10	**Orchestration** — calls must happen **in a particular order**	Sequences them and transforms data between steps
11	**Partial failures** in a **distributed system** (programs on many machines cooperating over a network)	Detects failures, **reroutes** or returns **fallback responses** (safe defaults)
12	**Request transformation** — services expect different request formats	Converts each request to what the target expects
13	**Client-specific adaptations** — mobile and web need different things	Adapts responses per client *(taken to its conclusion → BFF)*
14	**Centralized logging and monitoring** is hard across many services	**One point for observability** (understanding a running system from its logs, metrics, traces)
15	**Long-running operations** shouldn't block the client	Manages them **asynchronously**; clients **check progress or get a callback**

Lesson 3 — Worked example: a library system

Library users, staff and management use three services: **CatalogueService** (book searches), **UserService** (profiles), **InventoryService** (stock levels). Each is a Java **interface** (method signatures, no implementation) — e.g. `CatalogueService` exposes `searchByTitle(String)` and `searchByAuthor(String)`.

The gateway

A separate class is the single point of contact. It mirrors the services' methods, but **each method delegates to the appropriate service** rather than handling the request itself:

```java
public class ApiGateway {
    private UserService userService;
    private CatalogueService catalogueService;
    private InventoryService inventoryService;

    public ApiGateway(UserService userService,
                      CatalogueService catalogueService,
                      InventoryService inventoryService) {
        this.userService = userService;
        this.catalogueService = catalogueService;
        this.inventoryService = inventoryService;
    }

    public User getUserDetails(String userId) {
        return userService.getUserDetails(userId);
    }

    public Book searchByTitle(String title) {
        return catalogueService.searchByTitle(title);
    }

    // ...additional methods as needed...
}
```

Take the shape, not the detail: gateway methods *delegate*, never *implement*. Once a gateway method contains **business rules** (domain logic, e.g. "members may borrow 5 books"), you're on the road to a **distributed monolith** — services so tangled they must change and deploy together. Real gateways wrap each delegation with auth, **timeouts**, **retries** and caching, but the delegation shape stays.

Lesson 4 — Problems the gateway itself introduces

Problem	Mitigation
1. Single point of failure — if it goes down nothing communicates; its availability *is* your availability	**High availability** and **redundancy** (spare copies); monitor it diligently
2. Potential bottleneck — every request passes through it	Keep it fast; **scale horizontally** (more copies, not a bigger machine); optimise its code
3. Complexity in the gateway — routing, security, transformation all accumulate	Keep responsibilities clear; **split into multiple gateways** per client type or per functionality
4. Maintenance overhead — routes and rules drift from reality as services evolve	**Automate**; use **CI/CD** (automatic build, test and release of every change)
5. Security risk — the entry point is a **prime target**	Robust security practices, keep it patched, monitor for **vulnerabilities**

> Problem 3 is the failure mode to name explicitly: the gateway is where business logic goes to hide. Once domain rules live there, every team must change it and it becomes the deployment bottleneck microservices were meant to eliminate. Keep it about **transport** (getting requests to the right place), not **domain** (the business problem).

Lesson 4 (cont.) — Performance implications

The costs the gateway imposes on every request:

Implication	Solution
1. Latency — an extra network **hop** on every request	Optimise; **caching and response compression**
2. Resource utilization — the gateway itself can consume a lot	Provision adequately; monitor; **scale as needed**
3. Content aggregation overhead — merging responses is resource-intensive	Optimise aggregation; use **asynchronous** calls so nothing blocks
4. Rate limiting (capping requests per client per time window) is hard **in distributed deployments** — several gateway **instances** must share each client's usage	Algorithms built for it: **token bucket** or **leaky bucket**, with a shared store
5. Load balancing overhead — avoid **hotspots** (one instance getting most traffic)	Smart LB algorithms; a **service mesh** for finer control *(Sidecar)*
6. Async operations at the gateway are complex	Optimise; give clients **clear status-check or callback** mechanisms

Token vs leaky bucket: a **token bucket** refills at a steady rate and each request spends a token — a full bucket permits a short burst; a **leaky bucket** queues requests and releases them at a fixed rate, smoothing bursts away. Details: § Rate limiting.

Verdict: with careful design, monitoring and optimisation the costs are manageable — and the alternative, every client knowing every service, is worse.

Lesson 5 — System design example: Instagram

The gateway's responsibilities here

Routing · **aggregation** into one response · **authentication & authorization** · **rate limiting** against abuse · **load balancing** across service instances · **logging and monitoring**.

The microservices behind it

- **User Management** — profiles, followers/following, settings
- **Photo Upload and Processing** — upload, filters, resizing per device
- **Feed** — the feed of photos from people you follow
- **Notification** — new followers, likes, comments
- **Search** — profiles, hashtags, locations
- **Comment** — comments on photos

Scenario A — uploading a photo *(sequential)*

1. User uploads via the app → request hits the gateway
2. Gateway **authenticates** the user and checks they're **authorized to upload** — two separate checks, both before any service is touched
3. Gateway routes to the Photo Upload and Processing Service, which processes the image and responds
4. Gateway returns a confirmation to the app

Scenario B — viewing the feed *(parallel — the interesting one)*

1. User opens the feed → request hits the gateway, which authenticates
2. Gateway makes **parallel requests** to the **Feed Service** (recent photos from people followed) and the **Notification Service** (new notifications)
3. Gateway **aggregates** them and returns a **single consolidated response**

Why the contrast matters

Upload is a single routed call. Feed is a **scatter-gather**: one client request fans out to parallel service calls and returns one merged response. That's where the gateway earns its latency cost: client-side, it would be several sequential **round trips** over a mobile network; via the gateway it's one round trip plus parallel calls inside the **data centre**, where inter-service latency is a fraction of latency to the phone.

Consolidation

API Gateway vs. BFF

One shared entry point optimised for all clients generically (risk: bottleneck / distributed monolith) vs. one backend per client type (risk: duplication and drift). Full comparison table: BFF Part 8. They're not mutually exclusive — a common setup is a thin shared gateway (**TLS** termination, coarse auth, rate limiting) with BFFs behind it.

When to use

- Multiple microservices **and** multiple client types
- Clients would otherwise make many round trips per screen
- You want cross-cutting concerns (auth, rate limiting, logging) in one place
- You expose services to untrusted external consumers

When to avoid

- Very few services — the gateway is pure overhead
- Purely internal service-to-service traffic — a **service mesh** fits better
- One client whose needs diverge sharply — consider BFF
- Ultra-low-latency paths where an extra hop is unacceptable

Interview soundbite (30 seconds)

> "An API Gateway is a single entry point in front of your microservices. Clients call it instead of services directly, and it handles routing, aggregation, authentication, rate limiting, caching and logging in one place. The big win is that clients don't need to know your service topology — which matters most for mobile and TV clients you can't force-update, because topology baked into them is frozen for years. The classic example is Instagram's feed: one request fans out in parallel to the feed and notification services and returns one response, beating several sequential round trips from the phone. Trade-offs: it's a single point of failure needing HA and redundancy, it adds a hop, rate limiting across distributed gateway instances needs token- or leaky-bucket algorithms, and it tends to accumulate business logic until it becomes a distributed monolith — so keep it about transport, not domain, and split it per client type or domain if it grows. If each client type needs a genuinely different API shape, that's Backends for Frontends."

Implementation

A. Runnable demo

Full source: `demos/ApiGatewayDemo.java`

```
cd demos && ./gradlew runCh03
```

The heart of it is scatter-gather with per-service fallbacks (`pool` is a **thread pool** of reusable workers; each `Future` is a placeholder for a result arriving later):

```
Future<String> fFeed    = pool.submit(() -> feed.fetch(userId));
Future<String> fNotifs  = pool.submit(() -> notifications.fetch(userId));
Future<String> fProfile = pool.submit(() -> profile.fetch(userId));

result.put("feed",          get(fFeed,    "feed-unavailable"));
result.put("notifications", get(fNotifs,  "no-notifications"));
result.put("profile",       get(fProfile, "profile-unavailable"));

private String get(Future<String> f, String fallback) {
    try { return f.get(1, TimeUnit.SECONDS); }
    catch (Exception e) { return fallback; }        // partial failure contained
}
```

All three calls are **submitted before any result is read** — that's what makes them parallel. Reading `fFeed` before submitting the others would **serialise** them.

B. What you should see

```
=== 1. Without a gateway: client calls each service itself ===
  3 round trips from the client: 288ms
  >> on mobile add ~150ms latency PER trip: ~738ms

=== 2. Gateway, sequential aggregation ===
  1 client round trip, 3 sequential server calls: 279ms

=== 3. Gateway, PARALLEL scatter-gather ===
  1 client round trip, 3 parallel server calls: 129ms
  >> ~the slowest service (120ms), not the sum

=== 4. Auth enforced once, at the edge ===
  rejected: 401 Unauthorized  (no service was touched)

=== 5. Partial failure -> graceful degradation ===
  {feed=feed-data, notifications=no-notifications, profile=profile-data}

=== 6. Rate limiting, also at the edge ===
  429 Too Many Requests after 5 requests
```

Compare 2 and 3: 279ms → 129ms — parallel aggregation costs the *slowest* service, not the sum. **Section 4**: an unauthenticated request (HTTP **401**) touched **zero** backend services — the security argument as an observable fact. (**HTTP status codes** are the three-digit codes on every web response: 401 = not authenticated, 429 = too many requests.)

C. Production — Spring Cloud Gateway

Spring Cloud Gateway is a ready-made gateway on **Spring**, the dominant Java framework, added via a **Maven** dependency (Maven: Java build tool that fetches libraries). In the YAML, `lb://` means "resolve via **service discovery**" (Service Discovery), **predicates** decide which requests a route matches, and **filters** modify or guard them.

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-gateway</artifactId>
</dependency>
```

```yaml
spring:
  cloud:
    gateway:
      routes:
        - id: orders
          uri: lb://order-service          # lb:// = resolve via service discovery
          predicates:
            - Path=/api/orders/**
          filters:
            - StripPrefix=2
            - name: CircuitBreaker
              args:
                name: orderCB
                fallbackUri: forward:/fallback/orders
            - name: RequestRateLimiter
              args:
                redis-rate-limiter.replenishRate: 100    # steady-state req/s
                redis-rate-limiter.burstCapacity: 200    # burst allowance
            - name: Retry
              args:
                retries: 2
                statuses: BAD_GATEWAY,SERVICE_UNAVAILABLE
                methods: GET                             # only idempotent methods

        - id: products
          uri: lb://product-service
          predicates:
            - Path=/api/products/**
            - Header=X-API-Version, v2                   # route by version
```

Global cross-cutting filter — auth on every route (`Mono` is a **reactive** type: one value delivered later without blocking a thread):

```java
@Component
public class AuthFilter implements GlobalFilter, Ordered {
    @Override
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {
        String token = exchange.getRequest().getHeaders().getFirst("Authorization");
        if (!tokenService.isValid(token)) {
            exchange.getResponse().setStatusCode(HttpStatus.UNAUTHORIZED);
            return exchange.getResponse().setComplete();      // never reaches a service
        }
        return chain.filter(exchange);
    }
    @Override public int getOrder() { return -100; }          // run early
}
```

Kubernetes-native alternative — an **Ingress** or **Gateway API** resource (Kubernetes objects describing how outside traffic enters the cluster; § Kubernetes basics) does routing, TLS and rate limiting with no gateway application to write or run. Prefer it when needs are routing-shaped rather than aggregation-shaped.

D. Implementation pitfalls

1. Sequential calls that should be parallel. The most common gateway performance bug: calling `.get()` on each future as you create it is sequential code that looks **concurrent**.

2. No per-call timeout in aggregation. One slow service makes the whole response slow. Bound each future (`f.get(1, TimeUnit.SECONDS)`) so the slowest dependency can't dictate your latency.

3. Business logic in the gateway. The distributed-monolith trap (Lesson 4). Keep it to transport: routing, auth, rate limiting, aggregation, observability.

4. In-memory rate limiting behind multiple gateway instances. Three **replicas** each allowing 100 req/s means the real limit is 300. Use a **shared store** — **Redis**, a fast in-memory key-value database — hence `redis-rate-limiter` above.

5. Retrying non-idempotent requests. `methods: GET` is deliberate: a gateway retrying a **POST** can create duplicate orders. An operation is **idempotent** if repeating it has the same effect as doing it once — see Retry.

6. Running the gateway as a single instance. It's a single point of failure by design: run at least two replicas behind a **load balancer**, with resource limits and monitoring like any critical service.

7. Losing trace context. The gateway is the ideal place to generate a **correlation ID** (a unique ID passed to every service a request touches, so logs can be joined up) and propagate it **downstream**. Skip it and you can't trace a request across services.

Related patterns

- **BFF** — one gateway per client type instead of one shared gateway
- **Circuit Breaker** — the gateway is a natural place to configure breaker rules per backend route
- **Service Discovery** — how the gateway finds service instances; the gateway is the classic server-side discovery router
- **Strangler Fig** — the migration facade is often literally an API Gateway
- **Sidecar** — gateway handles north-south (external, in and out of the system) traffic; a mesh of sidecars handles east-west (internal, service-to-service)